Python coding reference

A classroom-friendly language surface for flight, vision, decisions, loops, and safe missions.

NO INSTALL

AUTOMATIC WAITING

LOCAL TRANSLATION

SIM + REAL

FIRST-FLIGHT.PY

```
1  # First flight
2  take_off()
3  fly("forward", 1, 15)
4  land()
```

Write Python; Hopper runs the proven flight engine

The Python tab translates a focused classroom subset locally. Bluetooth, Wi-Fi vision, the simulator, Stop & Land, and the camera gallery still use the same app runtime.

SAFE-FIRST-MISSION.PY

```python
# Commands wait automatically.
take_off()

try:
    fly("forward", 1, 15)
    hover()
    take_photo()
finally:
    land()
```

WHAT THE APP HANDLES

Promise-returning drone and vision calls receive automatic waiting. Indentation becomes nested program structure. A syntax problem is reported with its Python line number before the motors start.

WHAT STUDENTS WRITE

Use four spaces for each level. End if, elif, else, while, for, def, try, except, and finally headers with a colon. Press Tab or Enter after a colon and the editor inserts the indentation.

IMPORTANT

Do not write await, import modules, or paste ordinary desktop Python libraries. This is Hopper Python: a deliberately small surface that runs inside the browser or desktop app.

Flight lifecycle and timed motion

Seconds, degrees, directions, and power percentages are explicit so missions are easy to read and review.

`take_off()`	Take off and wait until the aircraft is ready.
`land()`	Land and wait for the landing sequence.
`hover()`	Zero motion and hold for about one second.
`wait(seconds)`	Pause without blocking Stop & Land.
`fly(direction, seconds=1, power=15)`	up/down/left/right/forward/backward.
`rotate(degrees=0, direction="clockwise")`	Timed yaw; use clockwise or counterclockwise.
`flip(direction)`	forward/backward/left/right; allow safe clearance.
`set_axis(axis, power)`	Persistent pitch/roll/yaw/gaz until reset.
`reset_motion()`	Zero every motion axis without landing.
`emergency_cutoff()`	Immediate motor cutoff - emergency only.

State, photos, and accessories

Return values can be stored in variables and tested in decisions. Photo capture saves the current camera frame to the session gallery.

`battery_level()`	Battery percentage, or None before telemetry arrives.
`is_flying()`	True while hovering, flying, or flipping.
`is_landed()`	True when the controller reports landed.
`wait_for_battery_change()`	Pause until a new battery event arrives.
`take_photo()`	Store the current real or simulated camera view.
`open_grabber()`	Open the attached grabber accessory.
`close_grabber()`	Close the attached grabber accessory.
`fire_gun()`	Fire one BB from the attached cannon.
`key_pressed("ArrowUp")`	Read a live keyboard key state.
`stopped()`	True after the operator selects Stop & Land.

LOW-BATTERY-CHECK.PY

```
1  battery = battery_level()
2  if battery is not None and battery < 25:
3      land()
```

Threshold vision and saved photos

Threshold and coverage use percentages from 0 to 100. Each sees or scan command captures a fresh frame.

`scan_threshold(threshold=60, invert=False)`	Return white/black coverage and center result.
`sees_binary("white", 60, False, 10)`	True when coverage meets the final percent.
`binary_center("white", 60, False)`	True when the center pixel has the color.
`take_photo()`	Save exactly what the current camera sees.

WHITE-PAPER-SEARCH.PY

```
1  take_off()
2  while not stopped():
3      if binary_center("white", 60, False):
4          take_photo()
5          break
6      fly("forward", 0.4, 12)
7  land()
```

FRESH FRAME RULE

sees_binary() and binary_center() scan before deciding. scan_threshold() is useful when you want the full result in a variable.

COCO objects and custom labels

Confidence is a fraction from 0 to 1 in Python. The built-in detector draws boxes; Teachable Machine classifies the whole frame.

`load_object_model()`	Load the local COCO-SSD network.
`scan_objects(confidence=0.55)`	Return up to 10 detections; detect_objects is an alias.
`sees_object("person", confidence=0.55)`	Scan and return True when that label is found.
`object_coordinate("person", "x", 0.55)`	Last x or y value on a -100 to +100 scale.
`object_x("person", 0.55)`	Shortcut for the most recent horizontal coordinate.
`object_y("person", 0.55)`	Shortcut for the most recent vertical coordinate.

OBJECT-DECISIONS.PY

```python
if sees_object("person", confidence=0.45):
    x = object_x("person", 0.45)
    print(f"person x = {x}")

# Load model files in the UI first.
if sees_custom_label("red flag", 0.75):
    take_photo()
```

`scan_custom_model()`	Return predictions from the loaded model.
`sees_custom_label("red flag", 0.75)`	Scan and test one custom class.

AprilTags: detect, decide, and align

Hopper uses tag36h11 IDs 0 through 586. Use "any" when the specific ID does not matter.

`scan_april_tags()`	Return every visible tag and update overlays.
`sees_april_tag("any")`	Scan and return True for one or more tags.
`sees_april_tag(42)`	Scan for one tag36h11 ID.
`center_on_april_tag("any", 10, 5, 5, 3)`	Center x/y, align yaw, and stop after lost searches.

TAG-MISSION.PY

```python
take_off()
if sees_april_tag(7):
    centered = center_on_april_tag(7)
    if centered:
        take_photo()
else:
    print("Tag 7 not found")
land()
```

NAMED OPTIONS

center_on_april_tag(id=7, power=8, center_slack=4, angle_slack=5, lost_searches=4)

Variables, values, printing, and operators

A variable is created with =. Python uses True, False, and None; strings may use single or double quotes.

VALUES.PY

```
1  speed = 15
2  direction = "forward"
3  found = False
4
5  fly(direction, 1, speed)
6  found = sees_object("person")
7  print(f"Found: {found}")
```

`print(value)`	Write values to the program console.
`len(value)`	Number of items or characters.
`contains(collection, value)`	True when a list or string contains a value.
`abs / min / max / round`	Common number helpers.
`int / float / str / bool`	Convert one value to a new type.

COMPARISONS

== equal != not equal < <= > >= is None is not None

BOOLEAN LOGIC

and requires both; or requires either; not reverses True/False. Parentheses make combined decisions easier to read.

Make decisions with if, elif, and else

Only the first matching branch runs. Every nested line is indented four spaces.

BATTERY-DECISION.PY

```python
battery = battery_level()

if battery is None:
    print("Waiting for battery data")
elif battery < 25:
    print("Battery too low")
    land()
else:
    take_off()
```

VISION-DECISION.PY

```python
if sees_object("person", 0.5):
    hover()
else:
    fly("forward", 0.4, 12)
```

COMBINE-TESTS.PY

```python
if is_flying() and not stopped():
    take_photo()
```

COMMON ERROR

A missing colon or inconsistent indentation stops translation before the flight run begins. The console points to the Python line.

Repeat with for and safe while loops

Loops yield control between passes so the red Stop & Land button remains responsive.

FOR-LOOP.PY

```python
# Repeat exactly four times.
for step in range(4):
    fly("forward", 0.5, 12)
    rotate(90)
```

WHILE-LOOP.PY

```python
# Repeat until found or stopped.
while not stopped():
    if sees_object("stop sign"):
        hover()
        break
    fly("forward", 0.4, 10)
```

`range(4)`	0, 1, 2, 3
`range(2, 6)`	2, 3, 4, 5
`range(6, 0, -2)`	6, 4, 2

LOOP CONTROLS

break leaves the nearest loop. continue skips to the next pass. pass is a temporary empty line. Prefer while not stopped(): for open-ended flight searches.

Functions and guaranteed landing

Use def to name a reusable sequence. Hopper automatically waits when a student-defined function is called.

FUNCTIONS.PY

```python
def search_leg(seconds, power):
    fly("forward", seconds, power)
    hover()
    return sees_object("person")

take_off()
try:
    found = search_leg(1, 12)
    print(f"Found: {found}")
finally:
    land()
```

DEF

Function parameters are simple names. return sends a value back. Calls may appear in an assignment or decision.

TRY / FINALLY

The finally section runs after success or error. Put land() there when a mission could fail after takeoff.

EXCEPT

Use except Exception as error: to log a problem with print(error), then land in finally.

Quick reference: units, rules, and support

Keep this page nearby during a lab. The detailed pages explain every command listed here.

UNITS

Power and threshold: 0-100 percent Confidence: 0-1 fraction Time: seconds Rotation: degrees Vision x/y: -100 to +100

DIRECTIONS

fly: up, down, left, right, forward, backward flip: forward, backward, left, right rotate: clockwise, counterclockwise

PYTHON RULES

Four spaces per level. Colon after block headers. Calls stay on one line. No await or import. Use True, False, None, and snake_case command names.

FLIGHT + STATE

take_off land hover wait fly rotate flip set_axis reset_motion battery_level is_flying is_landed wait_for_battery_change stopped key_pressed

PHOTOS + ACCESSORIES

take_photo open_grabber close_grabber fire_gun emergency_cutoff

BASIC TOOLS

print len range contains abs min max round int float str bool

Source of truth: lib/python.ts, lib/drone.ts, lib/vision.ts, and lib/runtime.ts

VISION

scan_threshold sees_binary binary_center load_object_model scan_objects detect_objects sees_object object_coordinate object_x object_y scan_april_tags sees_april_tag center_on_april_tag scan_custom_model sees_custom_label

SAFETY

Use conservative power and short motion times. Test vision while landed. Keep the area clear. Stop & Land remains the operator control; emergency_cutoff is for immediate motor shutdown only.